



PHASE 2 | TASK 3

Gold: the Live Revenue Ticker

Paid events from Silver aggregated into 5-minute revenue windows, published only once each window is final



STEP 1

Build the Gold Stream

WHAT

Streamed from Silver, kept only payment_received events, set a 10-minute watermark on event_ts and grouped into 5-minute tumbling windows, counting orders_paid and summing revenue. Appended to atliq.streaming.gold_revenue_5min with its own checkpoint.

WHY

Watermarks do not travel between tables, so Gold declares its own. Append mode writes each window exactly once, after it can no longer change, so every row in Gold is final and safe to report on.

STEP 1 | Build the Gold Stream

```
189 @@ -189,20 +189,76 @@
190 # COMMAND -----
191
192 - # TODO: Task 2 – your Silver stream here
193
194 + gold_df = (
195 +     spark.readStream.table(SILVER)
196 +     .filter(F.col("event_type") == "payment_received") # revenue = paid events only
197 +     .withWatermark("event_ts", "10 minutes") # watermarks do not travel between tables, so
198 +     set it again
199 +     .groupBy(F.window("event_ts", "5 minutes")) # tumbling: fixed, non-overlapping 5-min
200 +     buckets
201 +     .agg(
202 +         F.count("*").alias("orders_paid"),
203 +         F.sum("order_amount").alias("revenue"),
204 +     )
205 +     .select(
206 +         F.col("window.start").alias("window_start"),
207 +         F.col("window.end").alias("window_end"),
208 +         "orders_paid", "revenue",
209 +     )
210 + )
211 +
212 + gold_q = (
213 +     gold_df.writeStream
214 +     .outputMode("append") # a window is written once, only after it closes
215 +     .option("checkpointLocation", f"{CKPT}/{gold}") # Gold's own checkpoint
216 +     .trigger(availableNow=True)
217 +     .toTable(GOLD)
218 + )
219 + gold_q.awaitTermination()
220 + print("Gold windows:", spark.table(GOLD).count())
221 +
222 + # COMMAND -----
223 +
224 + # MAGIC %md
225 + # MAGIC ### Lag Query
226 +
```

Commit f9d6116: the Gold TODO replaced by the windowed aggregation with a separate checkpoint.



STEP 2

First Run, Two Minutes into a Live Burst

WHAT

Started a 20-minute producer run, then ran Bronze, Silver and Gold two minutes in. Gold held 3 windows, and the latest closed window ended at 19:20 UTC while the clock read 20:27 UTC.

WHY

The 3 windows are from earlier runs, long closed, so they were published at once. The windows for events arriving right now were held back: events kept landing in Silver, but none of the new windows was final yet.

STEP 2 | First Run, Two Minutes into a Live Burst (1 of 2)

Databricks

New

HomeLearnWorkspaceRecentsCatalogJobs & PipelinesComputeDiscoverMarketplaceSQLSQL EditorQueriesDashboardsGenie AgentsAlertsQuery HistorySQL WarehousesData EngineeringRunsData IngestionVisual Data PrepAI/MLPlaygroundAgentsAI GatewayExperimentsFeaturesModelsServing

Home

Tree view: ON

Type to search

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

FileEditViewRunHelpPythonFullLast edit was 5 minutes ago

TASK 3 — Gold: the live revenue ticker

From the Silver stream, keep only payment_received events and aggregate into 5-minute tumbling windows: orders_paid = count, revenue = sum of order_amount. Append closed windows to atliq.streaming.gold_revenue_5min.

Hint: F.window("event_ts", "5 minutes") — and think about WHY a window only appears after the watermark passes its end (you will explain this in your write-up).

Just now (29s)

14

Python

gold_df = (
 spark.readStream.table(SILVER)
 .filter(F.col("event_type") == "payment_received") # revenue = paid events only
 .withWatermark("event_ts", "10 minutes") # watermarks do not travel between tables, so set it again
 .groupBy(F.window("event_ts", "5 minutes")) # tumbling: fixed, non-overlapping 5-min buckets
 .agg(
 F.count("*").alias("orders_paid"),
 F.sum("order_amount").alias("revenue"),
)
 .select(
 F.col("window.start").alias("window_start"),
 F.col("window.end").alias("window_end"),
 "orders_paid", "revenue",
)
)

gold_q = (
 gold_df.writeStream
 .outputMode("append") # a window is written once, only after it closes
 .option("checkpointLocation", f"{CKPT}/{gold}") # Gold's own checkpoint
 .trigger(availableNow=True)
 .toTable(GOLD)
)
gold_q.awaitTermination()
print("Gold windows:", spark.table(GOLD).count())

> See performance (2)

> gold_df: pyspark.sql.connect.dataframe.DataFrame = [window_start: timestamp, window_end: timestamp ... 2 more fields]

Gold windows: 3

Lag Query

Two minutes in: Gold windows 3.

STEP 2 | First Run, Two Minutes into a Live Burst (2 of 2)

Free Edition

+ New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Tree view: ON

←

~/ ashish@alp...

📄

🔄

⬆️

⋮

Type to search

📁 01_kafka_stream_processing

📁 Bakehouse Sales Starter Space

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was 6 minutes ago

▶ 1 minute ago (29s)

14

```

)
)

gold_q = (
    gold_df.writeStream
        .outputMode("append")
        .option("checkpointLocation", f"{CKPT}/{gold}") # a window is written once, only after it closes
        .trigger(availableNow=True) # Gold's own checkpoint
        .toTable(GOLD)
)
gold_q.awaitTermination()
print("Gold windows:", spark.table(GOLD).count())

```

▶ [See performance \(2\)](#)

▶ gold_df: pyspark.sql.connect.dataframe.DataFrame = [window_start: timestamp, window_end: timestamp ... 2 more fields]

Gold windows: 3

Optimize

▶ Just now (5s)

16

SQL

🗑️

🔗

📄

⋮

```

%sql
SELECT MAX(window_end) AS latest_closed_window, current_timestamp() AS now
FROM atliq.streaming.gold_revenue_5min;

```

▶ [See performance \(1\)](#)

✓ Checked for improvements

Table

+

📄 latest_closed_window

📄 now

1	2026-10-01T19:20:00.000+00:00	2026-10-01T20:27:13.045+00:00
---	-------------------------------	-------------------------------

📄

1 row

5.340s runtime

Refreshed at 10/2/26, 1:57 AM (Now)

ⓘ

This result is stored as `_sql1df` and can be used in other Python and SQL cells.

Latest closed window 19:20 UTC versus now 20:27 UTC: live windows not yet published.



STEP 3

After the Burst: Windows Close Behind the Watermark

WHAT

After the producer finished, ran the three streams again. Gold grew to 5 windows; the latest closed window moved to 20:25 UTC, with the clock at 20:46 UTC.

WHY

A window closes only when the watermark (latest event time minus 10 minutes) passes its end, and with `availableNow` it is written on the run after the one that moved the watermark. So Gold trails real time by 10 to 20 minutes; the newest windows sit in Gold's streaming state until the next events arrive. That delay is the price of correctness, not lag.

STEP 3 | After the Burst: Windows Close Behind the Watermark (1 of 2)

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Tree view: ON

ashish@alp...

Type to search

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was 24 minutes ago

TASK 3 — Gold: the live revenue ticker

From the Silver stream, keep only `payment_received` events and aggregate into **5-minute tumbling windows**: `orders_paid` = count, `revenue` = sum of `order_amount`. Append closed windows to `atliq.streaming.gold_revenue_5min`.

Hint: `F.window("event_ts", "5 minutes")` — and think about WHY a window only appears after the watermark passes its end (you will explain this in your write-up).

Just now (27s)

14

Python

```
gold_df = (
    spark.readStream.table(SILVER)
    .filter(F.col("event_type") == "payment_received") # revenue = paid events only
    .withWatermark("event_ts", "10 minutes") # watermarks do not travel between tables, so set it again
    .groupBy(F.window("event_ts", "5 minutes")) # tumbling: fixed, non-overlapping 5-min buckets
    .agg(
        F.count("*").alias("orders_paid"),
        F.sum("order_amount").alias("revenue"),
    )
    .select(
        F.col("window.start").alias("window_start"),
        F.col("window.end").alias("window_end"),
        "orders_paid", "revenue",
    )
)

gold_q = (
    gold_df.writeStream
    .outputMode("append") # a window is written once, only after it closes
    .option("checkpointLocation", f"{CKPT}/gold") # Gold's own checkpoint
    .trigger(availableNow=True)
    .toTable(GOLD)
)

gold_q.awaitTermination()
print("Gold windows:", spark.table(GOLD).count())
```

> See performance (2)

✓ Checked for improvements

gold_df: pyspark.sql.connect.dataframe.DataFrame

window_start: timestamp

window_end: timestamp

orders_paid: long

revenue: decimal(22,2)

Gold windows: 5

After the burst: Gold windows 5.

STEP 3 | After the Burst: Windows Close Behind the Watermark (2 of 2)

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Home

Tree view: ON

Type to search

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was 25 minutes ago

```
2 minutes ago (27s)
14

gold_q.writeStream(
    .outputMode("append")                # a window is written once, only after it closes
    .option("checkpointLocation", f"{CKPT}/gold") # Gold's own checkpoint
    .trigger(availableNow=True)
    .toTable(GOLD)
)

gold_q.awaitTermination()
print("Gold windows:", spark.table(GOLD).count())
```

See performance (2) ✓ Checked for improvements

gold_df: pyspark.sql.connect.dataframe.DataFrame

```

window_start: timestamp
window_end: timestamp
orders_paid: long
revenue: decimal(22,2)
```

Gold windows: 5

Lag Query

Just now (2s)

16

SQL

See performance (1) ✓ Checked for improvements

Table

	latest_closed_window	now
1	2026-10-01T20:25:00.000+00:00	2026-10-01T20:46:27.060+00:00

1 row

1.716s runtime

Refreshed at 10/2/26, 2:16 AM (Now)

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

Latest closed window 20:25 UTC versus now 20:46 UTC: windows published only once final.



STEP 4

Read the Revenue Ticker

WHAT

Queried `gold_revenue_5min`: one row per closed 5-minute window with `orders_paid` and revenue, across the Task 1 run, the Task 2 top-up and tonight's burst.

WHY

The windows line up with each producer run: 07:50 and 07:55 hold the 281 payments from Task 1, 19:15 holds the 104 from Task 2, and 20:15 and 20:20 are the first closed windows of tonight's burst. A business user reads revenue per 5 minutes without touching raw events.

STEP 4 | Read the Revenue Ticker

databricks

Free Edition

+ New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Home

Tree view: ON

ashish@alp...

Type to search

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was 27 minutes ago

Run all Serverless Schedule Comments Share

Just now (2s) 18

SQL

Checked for improvements

See performance (1)

Table

	window_start	window_end	orders_paid	.00 revenue
1	2026-10-01T20:20:00.000+00:00	2026-10-01T20:25:00.000+00:00	269	745748.00
2	2026-10-01T20:15:00.000+00:00	2026-10-01T20:20:00.000+00:00	23	82653.00
3	2026-10-01T19:15:00.000+00:00	2026-10-01T19:20:00.000+00:00	104	236259.00
4	2026-10-01T07:55:00.000+00:00	2026-10-01T08:00:00.000+00:00	25	76654.00
5	2026-10-01T07:50:00.000+00:00	2026-10-01T07:55:00.000+00:00	256	684038.00

5 rows 1.614s runtime

Refreshed at 10/2/26, 2:18 AM (Now)

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Just now (2s) 19

SQL

Checked for improvements

See performance (1)

Table

	event_type	events
1	order_cancelled	323
2	order_shipped	1158
3	order_placed	1748
4	payment_received	1445

4 rows 1.827s runtime

Refreshed at 10/2/26, 2:17 AM (Now)

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

gold_revenue_5min: closed windows with orders_paid and revenue, next to Silver's event counts.



STEP 5

Reconcile Gold Against Silver

WHAT

Recomputed revenue per 5-minute window directly from Silver's payment_received events and compared it with Gold, window by window.

WHY

Every closed window matches to the paisa: diff 0.00 on all five rows. The same reconciliation habit as Phase 1, where Gold was checked against its source before anyone was allowed to trust it.

STEP 5 | Reconcile Gold Against Silver

databricks

Free Edition

+ New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Home

Tree view: ON

ashish@alp...

Type to search

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was now

Reconcile Gold Against Silver

Run all Serverless Schedule Comments Share

SQL

Just now (3s) 21

```
%sql
SELECT g.window_start, g.orders_paid,
       g.revenue AS gold_revenue, s.revenue AS silver_revenue,
       g.revenue - s.revenue AS diff
FROM   atliq.streaming.gold_revenue_5min g
JOIN   (
  SELECT window(event_ts, '5 minutes').start AS window_start,
         SUM(order_amount) AS revenue
  FROM   atliq.streaming.silver_order_events
  WHERE  event_type = 'payment_received'
  GROUP BY 1
) s ON g.window_start = s.window_start
ORDER BY g.window_start DESC;
```

See performance (1) Checked for improvements

Table

	window_start	orders_paid	.00 gold_revenue	.00 silver_revenue	.00 diff
1	2026-10-01T20:20:00.000+00:...	269	745748.00	745748.00	0.00
2	2026-10-01T20:15:00.000+00:...	23	82653.00	82653.00	0.00
3	2026-10-01T19:15:00.000+00:...	104	236259.00	236259.00	0.00
4	2026-10-01T07:55:00.000+00:...	25	76654.00	76654.00	0.00
5	2026-10-01T07:50:00.000+00:...	256	684038.00	684038.00	0.00

5 rows 2.804s runtime Refreshed at 10/2/26, 2:20 AM (Now)

This result is stored as _sqlidf and can be used in other Python and SQL cells.

Gold versus Silver revenue per window: diff 0.00 on every row.



STEP 6

Three Streams, Three Checkpoints

WHAT

The checkpoints volume now holds one folder per stream, bronze, silver and gold, next to the three tables in atliq.streaming.

WHY

Each stream keeps its own bookmark, so any one can be restarted, fixed or reset without disturbing the others. Sharing a checkpoint between streams would corrupt both; the guide grades this separation explicitly.

STEP 6 | Three Streams, Three Checkpoints

data

bricks

First Edition

+ New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Catalog

Serverless Starter Warehouse Serverless 2XS

Catalog

Type to search

My organization

workspace

default

information_schema

system

atliq

default

information_schema

streaming

Tables (3)

bronze_order_events

gold_revenue_5min

silver_order_events

Volumes (1)

checkpoints

Shares received

samples

Catalog Explorer > atliq > streaming >

checkpoints

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/atliq/streaming/checkpoints

Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
bronze		
gold		
silver		

About this volume

Owner ashish@alphainterbiz.com

Tags

Add tags

/Volumes/atliq/streaming/checkpoints: bronze, gold and silver folders; three tables in atliq.streaming.

Task 3 Complete

What this task delivered

Live revenue ticker

Paid events rolled into 5-minute tumbling windows in `gold_revenue_5min`, each written once, only when final.

Lateness made visible

Captured before and after a live burst: windows published 10 to 20 minutes behind real time, exactly as the watermark dictates.

Reconciled and separated

Gold matches Silver to the paisa in every window; three streams run on three separate checkpoints.

Proof

5

closed windows

0.00

Gold vs Silver diff

10 min

watermark

3

separate checkpoints

Engineering decisions: Watermark re-declared on Gold because it does not carry across tables; append mode so only final windows are written; tumbling 5-minute windows for a ticker that never double counts; `availableNow` on serverless, which means a window closes on the run after the watermark passes it; reconciliation against Silver before trusting Gold.